

专业实训与技能达标实验报告

组号	待填写
组成员	李天灿：学号/班级（待填写），GitHub：xgmx2005 / LiTiancan 陈颜西：学号/班级（待填写），GitHub：cyx-dawang
实验日期	2026 年 7 月 12 日
课程名称	专业实训与技能达标
实验名称（选题名称）	膳哉：智能菜谱助手
项目公开地址	https://github.com/xgmx2005/shanzai-recipe.git

说明：组号、成员姓名、学号和班级请在提交前替换为真实信息。

一、课程名称：专业实训与技能达标

二、实验名称（选题名称）：膳哉：智能菜谱助手

三、实验内容（选题具体要求）

本组选题为“智能菜谱助手”，属于创意娱乐类应用。系统面向希望节省做饭决策时间、关注饮食健康、希望根据现有食材安排餐食的用户，提供健康档案管理、对话式推荐、菜谱知识库、菜谱详情、收藏、购物清单和维护端管理等功能。

课程要求最终提交一个可运行的软件作品、一份实验报告和一个演示视频。本项目以 Web 应用形式实现，前端使用 Vue 3，后端使用 Spring Boot 3，数据库使用 MySQL 8.0，并通过 GitHub 完成协作和版本管理。项目公开仓库地址为：<https://github.com/xgmx2005/shanzai-recipe.git>。

四、报告正文

（一）实验目的

本实验的目标是完成一个接近真实产品流程的智能菜谱推荐系统，而不仅仅是静态展示菜谱。系统以“健康档案 + 食材输入 + 知识库推荐 + AI 解释 + 购物清单”为主线，帮助用户把“今天吃什么”转化为可执行的一餐方案。

通过本项目训练需求分析、数据库设计、前后端分离开发、REST API 设计、权限控制、Git 协作、AI 接口调用、用户体验设计和测试验证等综合能力。项目同时体现软件工程流程：先明确目标人群和功能边界，再设计数据库和接口，最后逐步完成页面、交互、后端逻辑和验收测试。

（二）成员分工与团队协作情况

成员	主要任务	占比	产出
李天灿	产品规划、需求分析、数据库设计、后端接口、推荐算法、AI 接入、前后端整合、页面体验优化、Git 合并与测试	约 85%	Spring Boot 后端、MySQL 脚本、DeepSeek 推荐解释、核心前端闭环、接口文档、测试验证、实验报告
陈颜西	前端脚手架、基础布局、首批页面实现、前后端接口初次对接、视觉风格探索	约 15%	Vue 前端基础布局、登录/用户端首批页面、前后端 API 对接初版

团队使用 GitHub 作为代码托管平台，采用 main 稳定分支和功能分支开发方式。提交信息统一使用中文，按照 feat、fix、docs、style、refactor、test 等类型描述具体动作。开发过程中每完成一个较清晰的功能点就进行提交，便于回溯问题和展示贡献。根据 Git 作者统计，李天灿对应账号 xgm2005 / LiTiancan 共 142 次提交，陈颜西对应账号 cyx-dawang 共 3 次提交，另有早期项目初始化账号 L2463323447 共 3 次提交。

项目开发后期多次进行本地验证，包括后端 Maven 测试、前端 Vitest 单元测试和 Vite 生产构建。最近一次主要验证结果为：后端 155 个测试通过，前端 54 个测试通过，前端生产构建成功。

Git 协作示例

贡献统计: xgmx2005/LiTiancan (李天灿) 142 次, cyx-dawang (陈颜西) 3 次, L2463323447 (初始化账号) 3 次
44ff677 fix: 调整注销账号入口位置
0ccebf0 merge: 合并账号注销功能
7b2de53 feat: 添加生产级账号注销流程
c247ad7 merge: 合并用户端体验收尾分支
2993b67 feat: 优化管理员知识库维护看板
49dd488 feat: 重构首页为今日饮食工作台

(三) 需求分析、数据库设计、技术选型与实现思路

1. 需求分析

用户角色	核心需求	本系统实现
普通用户	登录注册、填写健康档案、输入食材与饮食条件、查看推荐和详情	用户端工作台、健康档案、对话式智能推荐、推荐结果页、菜谱详情页
普通用户	收藏菜谱、生成购物清单、查看历史记录	收藏菜谱、推荐历史、可勾选购物清单、按多道菜生成合并清单
数据维护员	维护菜谱、食材和统计信息	维护端看板、菜谱维护、食材维护和核心统计数据
系统	规则稳定、AI 可解释、数据可追溯	知识库推荐、DeepSeek 推荐理由、推荐历史、兜底规则和异常提示

项目的核心需求不是“随机生成一个菜谱”，而是从稳定的菜谱知识库中选择合适菜谱。这样可以保证菜谱图片、食材用量、步骤、营养数据和购物清单都可追溯，避免 AI 随机生成带来的图片不匹配、热量不可靠、食材不可购买等问题。AI 在本系统中的定位是解释和归纳推荐理由，而不是随意创造数据库外的菜谱。

2. 技术选型

- 前端：Vue 3 + Vite + TypeScript + Naive UI + Pinia + Axios。Vue 负责组件化开发，Pinia 维护登录状态和用户信息，Axios 负责与后端 API 通信。

- 后端：Spring Boot 3 + MyBatis-Plus + Spring Security + JWT。Spring Boot 提供接口服务，MyBatis-Plus 简化数据库访问，JWT 实现前后端分离鉴权。
- 数据库：MySQL 8.0，使用 utf8mb4 字符集，支持中文菜谱、食材、用户昵称和 JSON 字段。
- AI 接入：DeepSeek Chat Completions API。AI 负责生成推荐理由、健康提示和购物建议；若 API Key 缺失或调用失败，系统使用规则兜底文案。
- 工程化：Maven 管理后端依赖，npm 管理前端依赖，GitHub 进行版本管理，Vitest/JUnit 进行单元测试。

3. 数据库设计

数据表	作用	关键字段/设计
user	保存普通用户和维护员账号	username、password_hash、role、status、avatar_url
user_profile	保存健康档案	身高、体重、BMI、饮食目标、忌口、过敏、烹饪时间
ingredient	食材知识库	分类、单位、每 100g 营养、别名 aliases
recipe	菜谱知识库	图片、步骤、营养、标签、目标人群、状态
recipe_ingredient	菜谱与食材关联	用量、单位、是否核心食材
recommendation_conversation/message	对话式推荐状态	阶段、上下文 JSON、消息记录、无效回答次数
recommendation_history/log	推荐历史与评分日志	输入条件、结果菜谱、AI 分析、分数
favorite	用户收藏	user_id + recipe_id 唯一约束
shopping_list/item	购物清单	多菜谱合并、食材数量、勾选状态

数据库围绕“建档案、输食材、推荐菜谱、看详情、生成购物清单、收藏和记录历史”的闭环设计。第一版没有把标签、目标人群等字段过度拆表，而是在保证可查询和可维护的前提下控制表数量，适合两人小组在短学期内完成。

4. 实现思路

- 1. 用户注册或登录后进入系统，JWT 存储在前端状态中，后续接口通过 Authorization 请求头访问。
- 2. 用户完善健康档案，包括身高、体重、BMI、饮食目标、忌口、过敏食材和烹饪时间偏好。
- 3. 智能推荐页通过类似 Agent 的对话方式询问用户想吃什么、已有食材、忌口、人数和时间，并允许手动修改识别到的条件。
- 4. 后端根据健康档案和输入条件构建推荐请求，先进行忌口和过敏硬过滤，再按照食材匹配、饮食目标、口味、烹饪时间和热度打分。
- 5. DeepSeek 根据推荐结果生成更自然的推荐解释；失败时返回后端规则兜底解释。
- 6. 用户可进入详情页查看做法步骤、营养和食材清单，也可把单道菜或多道菜加入菜单篮并生成购物清单。

（四）功能模块简介和系统结构图

模块	主要功能	对应实现
认证与账号模块	注册、登录、JWT 鉴权、头像/昵称维护、账号注销	auth、security、Pinia auth store
健康档案模块	BMI、目标热量、饮食目标、忌口/过敏/偏好	profile、ProfileView、ProfileOnboardingView
智能推荐模块	对话式收集条件、规则评分、AI 推荐说明、历史记录	recommendation、conversation、DeepSeek client
菜谱知识库模块	菜谱列表、详情、步骤、营养、真实图片	recipe、recipe_ingredient、Pexels 图片资源
购物清单模块	多菜谱生成清单、扣除已有食材、勾选采购	shopping、ShoppingListsView

收藏模块	收藏/取消收藏、收藏页展示、 账号绑定	favorite、FavoritesView
维护端模块	食材维护、菜谱维护、统计看 板	admin、admin views

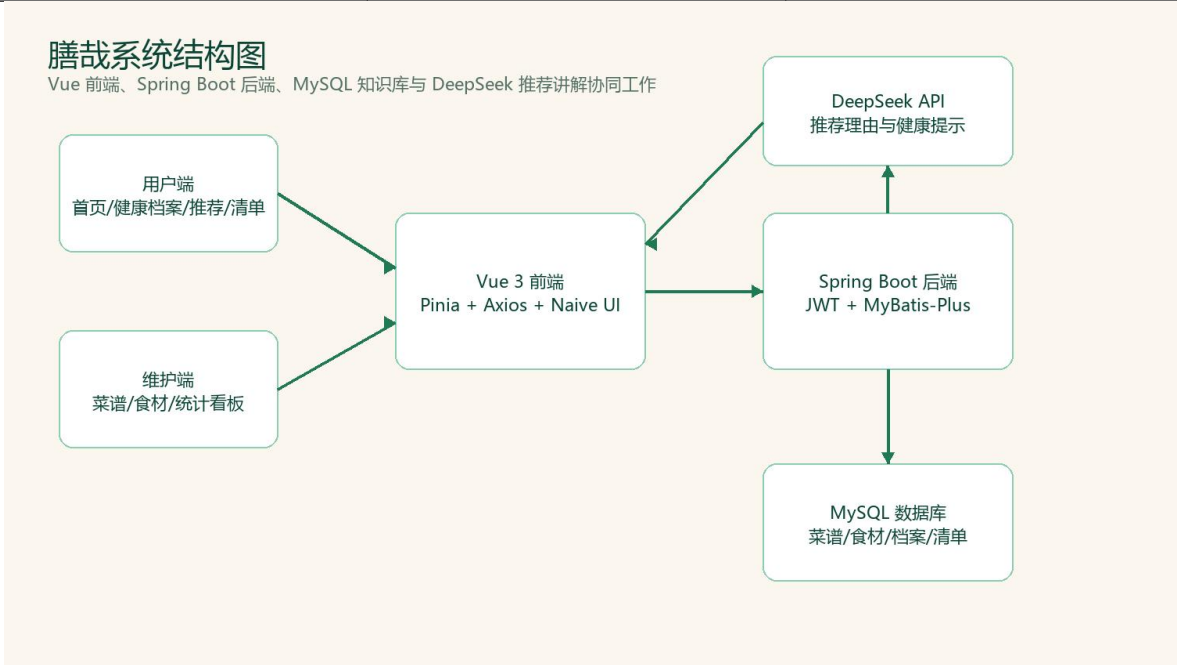


图 1 系统结构图：前端、后端、数据库与 AI 服务协同

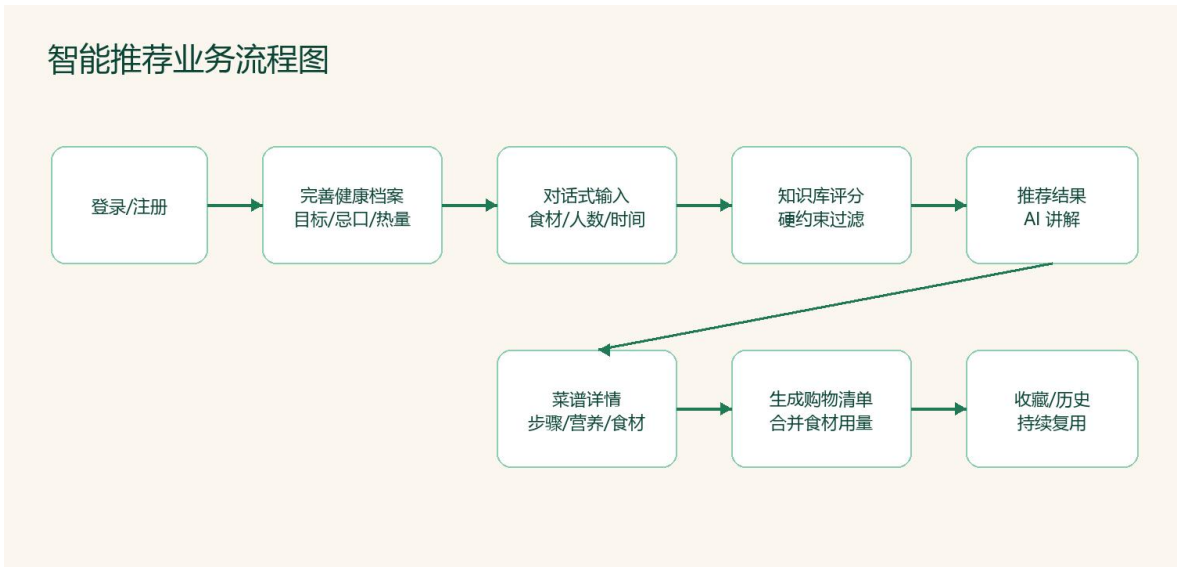


图 2 智能推荐业务流程图

（五）系统主要界面设计及运行说明

系统用户端采用现代轻健康风格，主色为深绿色和浅米色，强调食物、健康、清爽和可信赖感。导航栏保留首页、智能推荐、购物清单三个高频入口，健康档案、收藏菜谱、推荐历史等低频入口归入右上角用户菜单。维护端则采用更偏管理后台的布局，突出菜谱、食材和统计信息的维护效率。

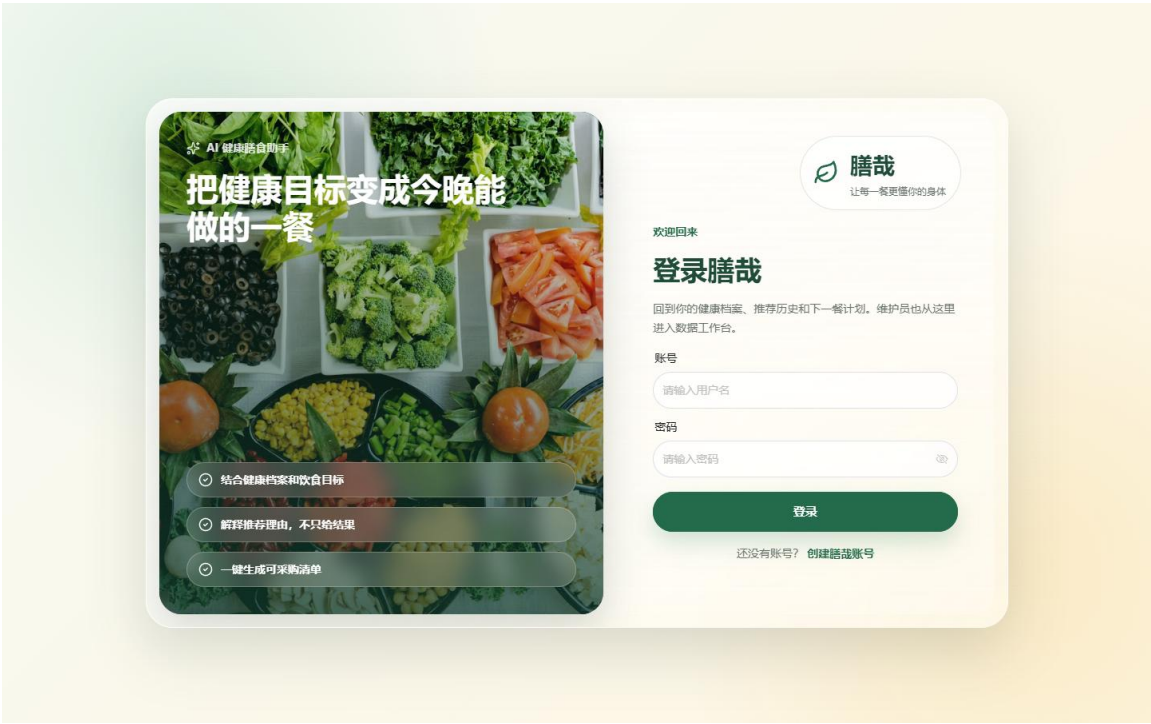


图 3 登录/注册入口：磨砂玻璃风格登录页，支持普通用户和维护员进入系统



图 4 用户首页：今日饮食工作台，展示推荐入口、健康状态和随机菜谱卡片



图 5 健康档案：维护身高体重、饮食目标、忌口和账号头像



图 6 智能推荐：对话式收集条件，识别饮食目标、时间、人数和食材



图 7 购物清单：按推荐菜谱生成可勾选采购清单

运行说明

1. 创建数据库：CREATE DATABASE shanzai_recipe DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci。
2. 导入脚本：依次执行 backend/src/main/resources/db/schema.sql 和 data.sql。
3. 后端启动：进入 backend 目录，执行 mvn spring-boot:run，默认端口为 8081。
4. 前端启动：进入 frontend 目录，执行 npm install 和 npm run dev，默认地址为 http://localhost:5173。
5. 普通用户演示账号：user1 / 123456；维护端账号：maintainer / 123456。
6. 普通用户流程：登录、完善健康档案、进入智能推荐、对话输入条件、查看推荐结果、进入菜谱详情、生成购物清单、收藏菜谱。
7. 维护端流程：使用 maintainer 登录后进入后台，查看统计看板，维护食材和菜谱数据。

（六）关键源程序代码

1. 推荐评分核心代码

```
public RecommendationScore score(RecipeCandidate candidate, RecommendationRequestModel request) {
```

```

        if (hasBlockedIngredient(candidate.ingredients(), request.excludedIngredients())
            || hasBlockedIngredient(candidate.ingredients(), request.blockedIngredients())) {
            return RecommendationScore.ineligible("包含忌口或过敏食材");
        }

        int ingredientScore = ingredientScore(candidateIngredients, available);
        int goalScore = goals.contains(clean(request.dietGoal())) ? GOAL_WEIGHT : 0;
        int tasteScore = intersects(tags, tastes) ? TASTE_WEIGHT : 0;
        int convenienceScore = convenienceScore(candidate.cookingTime(), request.cookingTime());
        int popularityScore = Math.min(POPULARITY_WEIGHT, Math.max(0, candidate.popularity()));

        int totalScore = ingredientScore + goalScore + tasteScore + convenienceScore +
            popularityScore;
        return new RecommendationScore(Math.min(100, totalScore), reasons, true);
    }
}

```

2. 对话式推荐消息处理代码

```

public ConversationResponse sendMessage(Long userId, Long conversationId,
    ConversationMessageRequest request) {
    synchronized (lockForConversation(conversationId)) {
        return Objects.requireNonNull(transactionTemplate.execute(
            status -> sendMessageLocked(userId, conversationId, request)));
    }
}

private ConversationResponse sendMessageLocked(Long userId, Long conversationId,
    ConversationMessageRequest request) {
    RecommendationConversationEntity conversation = requireConversationForUpdate(userId,
        conversationId);
    ConversationAnswerAnalysis analysis = interpreter.interpret(stage, userMessage.getContent(),
        context);
    ConversationTransition transition = flow.apply(stage, status, context, invalidAnswerCount,
        analysis);
    conversation.setStage(transition.stage().name());
    conversation.setStatus(transition.status().name());
    conversation.setContextJson(writeContext(transition.context()));
    conversationMapper.updateById(conversation);
    return getConversation(userId, conversationId);
}

```

3. 购物清单生成代码

```

@Transactional
public ShoppingListResponse createShoppingList(Long userId, ShoppingListCreateRequest request) {
    List<Long> recipeIds = cleanIds(request.recipeIds());
    if (recipeIds.isEmpty()) {
        throw new BusinessException("请选择菜谱");
    }
    List<RecipeEntity> recipes = loadActiveRecipes(recipeIds);
    List<RecipeIngredientEntity> rows = recipeIngredientMapper.selectList(

```

```
        new LambdaQueryWrapper<RecipeIngredientEntity>().in(RecipeIngredientEntity::getRecipeId,
recipeIds)
    );
    List<ShoppingNeed> shoppingNeeds = calculator.calculate(needs,
request.availableIngredients());
    ShoppingListEntity list = new ShoppingListEntity();
    list.setUserId(userId);
    list.setTitle(resolveTitle(request.title(), recipes));
    shoppingListMapper.insert(list);
    shoppingNeeds.forEach(need -> insertItem(list.getId(), need));
    return toResponse(list, listItems(list.getId()));
}
```

4. 前端鉴权请求示例

```
http.interceptors.request.use((config) => {
  const auth = useAuthStore()
  if (auth.token) {
    config.headers.Authorization = `Bearer ${auth.token}`
  }
  return config
})

export function deleteCurrentUser() {
  return http.delete<null>('/auth/me').then((res) => res.data)
}
```

（七）实验总结

1. 收获

本次课程设计最大的收获是把一个简单想法逐步拆成可实现的软件系统。最开始只是“根据食材推荐菜谱”，但在实现过程中发现真正可用的系统还需要健康档案、稳定菜谱知识库、真实图片、购物清单、收藏、历史记录、维护端和异常处理。通过持续拆分任务，我们把需求从创意落到了数据库表、接口、页面和测试上。

第二个收获是认识到 AI 功能不能简单等同于“让模型随便生成”。饮食和菜谱具有现实约束，如果 AI 生成一个数据库中不存在的菜谱，就会出现图片不匹配、食材用量无法计算、购物清单不可追溯等问题。因此本项目采用“知识库增强推荐”：推荐结果来自数据库，AI 负责解释推荐理由和健康提示。这种分层更适合课程项目，也更接近真实产品工程思路。

2. 创新点

- 采用对话式 Agent 推荐流程，而不是一次性堆叠很多表单项，用户可以自然表达“想吃清淡、高蛋白、30 分钟内完成”等条件。

- 把 AI 放在推荐解释层，菜谱实体、图片、营养和购物清单来自知识库，兼顾智能体验和数据可靠性。
- 健康档案影响推荐排序，BMI、目标热量、饮食目标、忌口和过敏共同参与计算。
- 推荐结果支持菜单篮，可选择多道菜合并生成购物清单，而不是只能生成单道菜清单。
- 用户端与维护端分离，维护员可管理菜谱和食材知识库，体现了系统长期维护能力。

3. 遇到的问题及解决方法

问题	原因	解决方法
MySQL 导入 SQL 失败	项目路径包含中文，MySQL 命令行 SOURCE 路径解析失败	改用 IDEA Database 控制台执行 schema.sql 和 data.sql，或复制到英文路径执行。
Maven 下载依赖不稳定	本机网络代理和镜像源状态不稳定	确认网络状态，必要时切换代理或 Maven 镜像，保证依赖完整下载。
AI 生成菜谱图片难以匹配	AI 随机生成菜谱和真实图片、食材数据无法一一对应	改为知识库菜谱 + 真实菜谱图片，AI 只生成推荐解释。
推荐输入界面占空间且不自然	标签表单难以覆盖用户真实表达	改为对话式输入，并加入条件识别、手动修改和错误提示。
购物清单体验冗余	早期页面统计和模式切换过多	精简为清单列表、当前清单、勾选状态和主题色进度条。
注销账号后旧 token 仍可能访问	只删除前端状态不足以让后端识别账号状态	后端 JWT 过滤器增加用户状态检查，注销时禁用账号并清理隐私数据。

4. 不足与改进方向

- 当前菜谱知识库数量仍然有限，后续可以继续扩充不同地区、不同季节、不同饮食目标的菜谱。
- DeepSeek 推荐解释目前依赖外部 API，正式上线时需要增加调用频率控制、成本控制和更完整的失败提示。
- 管理员端已具备基础维护能力，但还可以增加批量导入、图片审核、菜谱状态流转等功能。
- 目前部署主要面向本地演示，若上线给朋友使用，还需要完善 HTTPS、服务器部署、备份、安全审计和日志监控。

- 实验报告中的组号、成员真实姓名学号和演示视频截图需要在提交前补充完整。

五、得分：

实验指导教师：_____ 年 月 日